

AHCL 360°

Mobile App, Admin Panel & API

Project Report: Development Steps, Architecture and Run Guide

Employee Panel • Channel Partner Panel • Web Admin Panel • REST API

Stack: React Native (Expo SDK 57) • Node.js + Express 5 • MongoDB • React 19 + Vite

Report date: 7 October 2026

Project folder: C:\Users\ASUS\Desktop\Project

Contents

1. Executive Summary	3
Verification results at a glance	3
2. Requirements Covered	4
2.1 Employee Login Panel (mobile)	4
2.2 Channel Partner Login Panel (mobile)	4
2.3 Common features and admin panel	5
3. Architecture and Technology	6
3.1 System overview	6
3.2 Technology stack	6
3.3 Project folder structure	6
4. Development Steps Performed	8
Step 1: Requirement analysis and decisions	8
Step 2: Environment check	8
Step 3: Generated the project skeletons	8
Step 4: Built the backend API	8
Step 5: Tested the backend and fixed issues	8
Step 6: Built the web admin panel	9
Step 7: Built the mobile app	9
Step 8: Verified the mobile app	9
Step 9: End-to-end smoke test	9
Step 10: Documentation and project files	9
5. Backend Design Details	10
5.1 Security model	10
5.2 Key business rules	10
5.3 Offline sync design	10
5.4 API overview	11
6. Web Admin Panel	12
7. Mobile Application	13
7.1 Employee panel	13
7.2 Channel partner panel	13
7.3 Device behaviour	13
8. Testing and Quality Checks	14
8.1 Automated API tests (17)	14
8.2 Build, bundle and lint checks	14
9. How to Run the Project (Development)	15
9.1 Prerequisites	15
9.2 Start the backend API (Terminal 1)	15
9.3 Start the admin panel (Terminal 2)	15
9.4 Start the mobile app (Terminal 3)	15
9.5 Demo accounts	16
9.6 Optional: persistent database with Docker	16
9.7 Run the automated tests	16
9.8 Troubleshooting	16
10. Steps Required for Production	18
10.1 Backend configuration	18
10.2 Hosting	18
10.3 Push notifications (Firebase Cloud Messaging)	18
10.4 Mobile builds and store release	18
11. Known Limitations and Recommended Next Steps	20
Limitations	20
Recommended next steps	20

If the contents list is empty, right-click it in Word and choose "Update Field".

1. Executive Summary

AHCL 360° is a single platform for AHCL's field employees (sales, admin and field staff) and its channel partners (super distributors, distributors and stockists). This report describes what was built, every step taken to build and verify it, and the exact steps needed to run it on a development machine and take it to production.

The project was delivered as a **full working scaffold**: every module in the requirements document has a functioning first version backed by real API endpoints and data, not placeholder screens.

Component	Folder	What it contains
REST API	backend/	15 route modules, 30 MongoDB collections, OTP + JWT security, PDF/Excel reports, GST invoicing, offline sync, demo data, 17 automated tests
Web admin panel	admin/	15 pages: dashboard and charts, users, approvals, HR and payroll, targets, live tracking and route maps, geofences, distributors, retailers, products, orders and invoices, broadcast, support, reports
Mobile app	mobile/	42 route files covering the Employee panel and the Channel Partner panel, plus shared notifications, support, profile and offline-sync screens

Verification results at a glance

Check	Result
Backend automated test suite (npm test)	17 of 17 tests passed
Admin panel production build (vite build)	Builds successfully; main bundle reduced from 862 kB to 278 kB with code splitting
Admin panel → API smoke test (all endpoints the panel calls)	All returned HTTP 200; PDF and Excel reports downloaded correctly
Mobile app bundle (expo export for Android and iOS)	Both platforms compile
Mobile health check (expo-doctor)	21 of 21 checks passed
Mobile lint (expo lint)	Clean: 0 errors, 0 warnings
Visual check on a real phone / browser	Not yet done. This is the first recommended next step.

2. Requirements Covered

The requirements document defined three audiences and a set of common features. The table below maps each requirement to where it is implemented.

2.1 Employee Login Panel (mobile)

Requirement	Implementation
Secure login with OTP or password; role-based access	OTP (hashed, 5-minute expiry, attempt-limited) or password login. Roles: super_admin, admin, manager, field_executive, staff. Roles are enforced on the server for every request.
GPS live tracking, auto start/stop with office hours	Background location task records points only while the employee is checked in and within their office hours; it stops itself after hours. The server also discards points outside office hours.
Visit logs: client, purpose, timestamp, distance (km), check-in/out location validation	Visit check-in/check-out with GPS. Distance is calculated from the GPS trail since the previous visit. A visit is flagged if the employee is more than 300 m from the client's saved location.
Route map history	Map of the day's GPS trail with numbered visit markers (mobile and admin panel).
Daily check-in/out with GPS	Check-in/out validated against office geofences (configurable: flag only, or block).
Leave application (Pending/Approved/Rejected); holiday calendar	Leave requests with overlap check, leave balance, manager approval with comments; holiday calendar managed by admin.
Monthly attendance report (PDF)	Calendar view with present/half-day/absent/leave/holiday summary and PDF download.
Payslips, salary summary, document vault, KYC upload	Payslip PDFs, 12-month salary summary, HR-issued documents and employee uploads, KYC with encrypted PAN/Aadhaar/bank details.
Travel expense claim: bills, auto calculation by distance, status tracking	Bills as photo or PDF; amount = GPS distance × rate per km for the travel mode; Pending → Approved → Reimbursed.
Mapped distributors; stock in / out / closing / return with reason	Distributor-wise stock ledger with closing stock calculation; returns and adjustments require a reason.
Daily Sales Order (DSO); target vs achievement (monthly/quarterly)	DSO entry from the product catalog; monthly and quarterly targets for sales, visits and new retailers.
Team management: junior mapping, daily work reports, approve/comment, track juniors' GPS, performance matrix	Reporting hierarchy (multi-level), work reports with auto-attached stats, manager review, live team map, weighted performance score.
Auto-generated PDF/Excel reports	Daily Work Summary, Expense, Visit Route, Attendance and Sales reports in PDF and Excel.

2.2 Channel Partner Login Panel (mobile)

Requirement	Implementation
Login via mobile/email/OTP; tiers Distributor / Super Distributor / Stockist	Partner logins are linked to a distributor record. Super distributors can also see their child distributors.

Requirement	Implementation
Place orders (dynamic catalog); summary with unit, price, discount, total	Searchable catalog with categories and a quantity stepper. The server prices every order, so clients cannot change prices.
Track order status; download invoices (PDF)	Processing → Packed → Shipped → Delivered with history. Packing generates the GST invoice; delivery adds the goods to the partner's stock automatically.
Inventory: stock in/out, reorder suggestions, closing stock, adjustments	Stock ledger; reorder suggestions from the configured level or 30-day average sales; damaged/returned adjustments.
Invoice history; GST-compliant invoices	CGST + SGST for intra-state and IGST for inter-state supply, HSN codes, amount in words (Indian numbering).
Retailer onboarding: name, contact, GPS, KYC (PAN/Aadhaar/GST), linked to distributor code; list with filter/sort	Implemented with GPS capture, encrypted KYC numbers and KYC document uploads.
Claims and settlements with bills, real-time status, notifications	Pending / Approved / Rejected / Revision requested / Settled, with a resubmission flow and notifications.

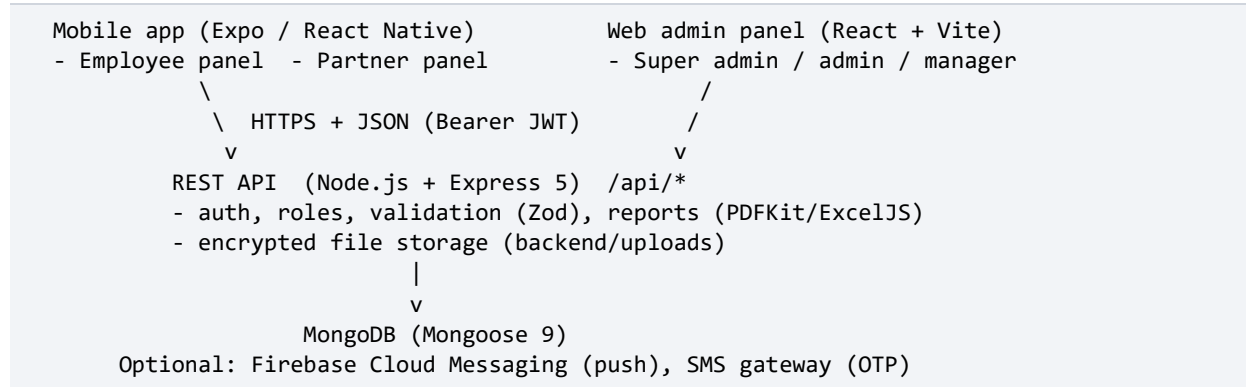
2.3 Common features and admin panel

Requirement	Implementation
Push notifications (orders, claims, attendance reminders, salary credit, targets/announcements)	Stored in-app for every event; sent through Firebase Cloud Messaging once configured. Automatic attendance and check-out reminders.
Settings & support: profile, change password, support tickets, call/email	Implemented in both panels; ticket conversation threads with the admin team.
Offline mode with automatic sync	Attendance, visits, GPS points, expenses, DSO, stock entries and work reports are queued offline and replayed when back online. Each entry is applied only once.
Security: encryption, auto logout after inactivity, admin block/reactivate	AES-256-GCM field and file encryption, idle timeout on server and clients, immediate session revocation on block.
Analytics dashboard with export	KPI tiles and charts in the admin panel; every report exportable to PDF/Excel.
Admin panel: onboarding/roles, approvals, targets/territories, distributor & retailer management, reports, broadcast, geofencing	All implemented in the web admin panel, plus an audit log of admin actions.

3. Architecture and Technology

3.1 System overview

The system has three applications that talk to one REST API. The API is the single source of truth and enforces every permission rule; the mobile app and the admin panel are clients only.



3.2 Technology stack

Layer	Technology (version used)	Why
Mobile	Expo SDK 57, React Native 0.86, React 19.2, Expo Router 57	One JavaScript codebase for Android and iOS; managed native modules for GPS, camera, notifications and secure storage
Mobile modules	expo-location, expo-task-manager, expo-secure-store, expo-notifications, expo-file-system, expo-image-picker, expo-document-picker, react-native-maps, NetInfo, AsyncStorage	Background GPS, keystore storage of tokens, push, downloads/sharing, bill/KYC capture, maps, offline detection
API	Node.js 24 (20+ supported), Express 5.2, Zod 4, jsonwebtoken, bcryptjs, multer, helmet, express-rate-limit	Same language as the clients; async error handling in Express 5; schema validation on every input
Database	MongoDB with Mongoose 9.11	Flexible documents for orders, ledgers and GPS points; aggregation pipelines for stock and reports
Documents	PDFKit 0.20, ExcelJS 4.4	Server-side PDF (invoices, payslips, reports) and Excel generation
Admin panel	React 19, Vite 8, React Router 7, Recharts 3, Leaflet + react-leaflet 5	Fast build, charts, OpenStreetMap maps for geofences and tracking
Dev database	mongodb-memory-server 11; Docker Compose (mongo:7)	API runs with zero setup; persistent DB with one command

3.3 Project folder structure

Project/	
├─ README.md	Quick start, feature map, security notes
├─ docker-compose.yml	Local MongoDB
├─ backend/	
│ └─ src/config/	env, database, roles
│ └─ src/models/	30 Mongoose collections
│ └─ src/middleware/	auth, validation, errors

```

|   ├── src/services/      tokens, tracking, stock, billing (GST), documents (PDF/Excel),
messaging, jobs
|   ├── src/routes/       15 route modules
|   ├── src/utils/        crypto, geo (distance), dates (IST), scope (permissions)
|   ├── src/seed.js       demo data
|   └── test/api.test.js   17 end-to-end tests
└── admin/
    ├── src/ lib/ (API client, hooks) components/ (UI kit, map) pages/ (15 pages)
└── mobile/
    ├── app.json          permissions, plugins (location, notifications, image picker)
    └── src/
        ├── app/          routes: login, employee/(tabs), partner/(tabs), common
        ├── components/   UI kit, product picker, sync banner
        └── lib/          api, auth, offline queue, background tracking, push, pickers

```

4. Development Steps Performed

This section records, in order, everything that was done to build the project, including problems met and how they were fixed.

Step 1: Requirement analysis and decisions

- Read the requirements document and confirmed two key decisions with the project owner: build a **full project scaffold** (not just a plan), using **React Native + Node.js + MongoDB**.
- Chose Expo (managed React Native) for the app, Express for the API and a React + Vite web panel for admins.

Step 2: Environment check

- Confirmed tools on the development PC: Node.js 24.13.1, npm 11.19, Git 2.53 (Windows 11).
- MongoDB was **not installed**. Decision: the API starts an in-memory MongoDB automatically in development, and a `docker-compose.yml` is provided for a persistent database (MongoDB Atlas also supported).

Step 3: Generated the project skeletons

- `npx create-expo-app@latest mobile --template blank` produced the Expo SDK 57 app.
- `npm create vite@latest admin -- --template react` produced the admin panel.
- Created the `backend/` folder structure by hand (config, models, middleware, services, routes, utils).

Step 4: Built the backend API

1. **Dependencies:** express, mongoose, jsonwebtoken, bcryptjs, multer, cors, helmet, morgan, dotenv, zod, pdfkit, exceljs, express-rate-limit; dev: mongodb-memory-server, supertest.
2. **Configuration** (`src/config`): all settings from environment variables with safe development defaults. Production refuses to start without real secrets.
3. **Utilities:** AES-256-GCM encryption for fields and files; haversine distance and a GPS-noise filter (ignores poor-accuracy fixes, jitter under 15 m and impossible jumps over 150 km/h); India-time (IST) date helpers; permission scoping (self, team or all; own distributor or child distributors).
4. **Data model:** 30 collections. Users, OTPs and refresh tokens; files; territories, geofences, GPS points, visits; attendance, leaves, holidays, payslips, documents; expenses; distributors, retailers, products, stock ledger, reorder settings, sales orders, targets; orders, invoices, counters; work reports, claims, tickets, notifications, sync log and audit log.
5. **Services:** token issue/rotation with idle timeout; geofence checks and GPS distance; stock ledger, closing stock and reorder suggestions; GST pricing and invoices (CGST/SGST vs IGST, amount in words); PDF/Excel report builder; notifications (in-app + FCM); OTP sending (SMS webhook or console); a scheduler for attendance and check-out reminders.
6. **Routes:** auth, files, attendance, tracking, visits, HR, expenses, sales (products, distributors, stock, DSO, targets), partner (orders, invoices, retailers, claims), team, reports, sync, analytics, common (profile, notifications, support) and admin.
7. **Demo data** (`seed.js`): 2 territories, an office geofence, 5 employees (super admin, admin, manager, 2 field executives), 3 distributors with partner logins, 7 products, opening stock and movements, retailers, orders in every status with invoices, 5 days of attendance, GPS trails, visits, sales orders, work reports, expenses, targets, holidays and payslips.

Step 5: Tested the backend and fixed issues

- Wrote 17 end-to-end tests (`test/api.test.js`) that start a fresh in-memory database, load the demo data and exercise the main flows of all three panels.

- All 17 passed on the first run. Two tests depended on the time of day (seeded attendance starts at 09:35 IST, and GPS points outside office hours are discarded). They were rewritten so they pass at any hour.
- Hardening during review: added a distributor access check to the analytics endpoint; the login response now includes the partner's distributor name, which the partner home screen needs.

Step 6: Built the web admin panel

- Added React Router, Recharts, Leaflet and react-leaflet. Configured the Vite dev server to proxy `/api` to the backend on port 4000.
- Wrote an API client that attaches the access token, refreshes it once when it expires, and logs out when the session is revoked or idle. Admins are also logged out locally after inactivity.
- Built a small UI kit (tables with pagination, forms driven by field definitions, modals, toasts, status badges) with light and dark themes.
- Built 15 pages (listed in Section 6). Charts follow data-visualisation best practice: one colour per single-series chart, a recessive grid and tooltips on every chart.
- Production build passed. Pages were then lazy-loaded, which reduced the main JavaScript bundle from 862 kB to 278 kB.

Step 7: Built the mobile app

- Read the official Expo SDK 57 documentation for Expo Router, Location, Notifications and FileSystem before writing code, because these APIs change between SDK versions (for example, the new `File.downloadFileAsync` download API and the new notification-handler fields).
- Installed all native modules with `npx expo install` so their versions match SDK 57.
- **Problem:** installing `@expo/vector-icons` failed with a peer-dependency conflict (`react-dom@19.3.0` versus Expo's `react@19.2.3`). **Fix:** pinned `react-dom` to the SDK version with `npx expo install react-dom`.
- Configured `app.json`: app name and IDs, deep-link scheme, location (including background), camera and photo permission texts, and the notifications, image-picker and location plugins.
- Wrote the core libraries: API client (tokens in the device keystore), auth context with inactivity logout, offline queue with automatic sync, background GPS task, push registration and file pickers.
- Built 42 route files: login, 5-tab Employee panel, 5-tab Partner panel, and shared screens.

Step 8: Verified the mobile app

- `npx expo export` for Android and iOS: both bundles compiled.
- `npx expo-doctor`: first run flagged a missing peer dependency (`expo-font`). It was installed and the second run passed 21/21.
- `npx expo lint`: found 6 issues (React Compiler purity rules in the auth provider, a state update inside an effect on the order screen, an unescaped apostrophe). All were fixed; lint is now clean.

Step 9: End-to-end smoke test

- Started the API and the admin dev server together and called every endpoint used by the admin panel and the partner app through the admin proxy. All returned HTTP 200.
- Downloaded an Expense report (Excel) and a Visit Route report (PDF); both returned valid files.

Step 10: Documentation and project files

- `README.md` with quick start, demo accounts, a feature-to-screen map, security notes and production steps.
- `.env.example` files for backend, admin and mobile; `docker-compose.yml` for MongoDB; `.gitignore` (excludes `node_modules`, `.env`, encrypted uploads and native build folders).
- Removed template leftovers (the Vite README and the Expo template licence) and the nested Git folder created by the Expo generator.

5. Backend Design Details

5.1 Security model

Area	How it works
Login	OTP: 6 digits, stored hashed, valid 5 minutes, max 5 attempts. Password: bcrypt (cost 12). Requests for unknown numbers get the same response, so accounts cannot be discovered. Auth endpoints are rate-limited.
Panel separation	A partner account cannot sign into the employee panel or the admin panel, and an employee cannot sign into the partner panel.
Sessions	Access token (JWT) lasts 15 minutes. Refresh tokens are random, stored hashed and rotated on every use; a reused token is rejected.
Auto logout	A refresh token unused for longer than <code>IDLE_TIMEOUT_MINUTES</code> (default 30) is rejected. The mobile app and admin panel also log out locally after inactivity.
Block / reactivate	Blocking a user, changing a password or forcing logout increments the user's token version, which invalidates every session immediately.
Data encryption	PAN, Aadhaar and bank account numbers are encrypted with AES-256-GCM in the database and masked in responses (e.g. *****9012). All uploaded files are encrypted on disk and served only through an authorised endpoint.
Authorisation	Every query is scoped: employees see their own data, managers see their full reporting tree, admins see everything; partners see their own distributor (super distributors also see child distributors).
Input validation	Every request body is validated with Zod (e.g. PAN, Aadhaar, GSTIN, IFSC formats).
Audit	User creation, updates, blocks, forced logouts, geofence changes and broadcasts are written to an audit log.

5.2 Key business rules

- **Attendance:** under 4 hours of work is a half day. Office geofences either flag or block out-of-fence check-ins.
- **GPS distance:** sum of distances between filtered GPS points; drops fixes with accuracy over 100 m, ignores jitter under 15 m and jumps over 150 km/h.
- **Visits:** only one open visit at a time; location is valid within 300 m of the client's registered point.
- **Expenses:** travel by bike or car = GPS km × rate (bike ₹3.5, car ₹9, public ₹2 per km, configurable); non-travel claims above ₹500 need a bill.
- **Stock:** the ledger stores signed movements; closing stock is the sum. A movement that would make stock negative is rejected; returns and adjustments need a reason.
- **Reorder suggestion:** reorder level = configured level, or max(product default, 30-day average daily sales × (7 lead days + 3 safety days)).
- **Orders:** priced on the server from the catalog; only admins can apply discounts; partners can cancel only while Processing.
- **Invoices:** created when an order is packed; numbered per Indian financial year (e.g. INV/2627/00001); intra-state uses CGST+SGST, inter-state uses IGST; rounded to the rupee with the amount in words.
- **Performance score:** 40% sales vs target + 25% visits vs target + 20% attendance + 15% daily reporting (each capped at 100%).
- **Leave quotas:** casual 12, sick 10, earned 15 days per year.

5.3 Offline sync design

When the phone has no internet, write actions are saved in a local queue with a unique client ID. When connectivity returns, the app sends the queue to [POST /api/sync](#). The server records every processed client ID, so an entry sent twice is applied once. Permanent errors (for example, a validation failure) are shown to the user on the Offline Sync screen; temporary server errors are retried.

5.4 API overview

Area	Base path	Main operations
Authentication	/api/auth	otp/request, otp/verify, login, refresh, logout, me, change-password, fcm-token
Attendance	/api/attendance	check-in, check-out, today, month, report (PDF/XLSX), team
Tracking	/api/tracking	config (office hours), pings (batch upload), route, live
Visits	/api/visits	create (check-in), check-out, list, detail
HR	/api/hr	leaves (+review, cancel, balance), holidays, payslips (+PDF), salary-summary, documents, kyc
Expenses	/api/expenses	estimate, create, list, update, review, reimburse
Sales	/api	products, distributors, stock, stock/txns, stock/reorder-suggestions, dso, targets
Partner	/api	orders (+quote, status), invoices (+PDF), retailers (+KYC review), claims (+review)
Team	/api/team	juniors, work-reports (+review), performance
Reports	/api/reports/:type	daily-work, expense, visit-route, attendance, sales, orders, stock (?format=pdf xlsx)
Other	/api	analytics/dashboard, analytics/charts, notifications, support/tickets, profile, files, sync
Admin	/api/admin	users (+block, force-logout), territories, geofences, broadcast, audit-logs

6. Web Admin Panel

The admin panel is used by super admins, admins and managers. Managers can view their own team's data; admin-only actions are rejected by the API for them.

Page	Purpose
Dashboard	Attendance, visits, sales and pending approvals for today; six-month charts for order value, DSO sales, visits, orders by status and top distributors.
Employees & Users	Onboard employees and partner logins, assign roles, managers, territories, office hours and salary; verify KYC; block/reactivate; force logout.
Approvals	Leaves, expense claims (approve, reject, reimburse), partner claims (approve, request revision, settle) and work reports; view attached bills.
HR & Payroll	Holiday calendar, payslip generation from salary structure (with salary-credit notification), document vault issue and verification.
Targets	Set monthly/quarterly targets and view the performance matrix and score.
Live Tracking & Routes	Live map of field staff (auto-refresh every 30 s), route history with visits, team attendance with geofence status.
Territories & Geofences	Create territories; draw geofences by clicking on the map; set radius and flag/block mode.
Distributors	Create and edit partners and their tiers; view closing stock, ledger, reorder suggestions and orders per distributor.
Retailers	Filter and sort retailers; view KYC documents and verify or reject KYC.
Products	Catalog with SKU, price, MRP, GST slab, HSN and reorder level.
Orders & Invoices	Move orders through their statuses, add tracking numbers, download GST invoices.
Broadcast	Send notifications to everyone or filter by user type, role or territory; see recent admin activity.
Support Tickets	Reply to tickets and change their status.
Reports	Download all seven report types as PDF or Excel for any date range, employee or distributor.

7. Mobile Application

7.1 Employee panel

Tab / screen	Features
Home	Check-in/check-out with GPS, tracking status, today's visits and month's sales, team summary for managers, target progress, quick actions, offline-sync banner.
Visits	Start/finish client visits, day-by-day history with distance and location validity, route map.
Sales	Mapped distributors (stock in/out/return, closing stock, reorder alerts, call and directions), daily sales orders, target progress bars.
HR	Attendance summary and calendar with PDF, leave balance and requests, holidays, payslips and salary summary, document vault, KYC.
More	Expenses (with GPS auto-calculation and bill capture), daily work report, reports download, team management for managers, notifications, profile, support, logout.
Team (managers)	Juniors' status today, live map, performance matrix, leave and expense approvals, work-report review with comments.

7.2 Channel partner panel

Tab / screen	Features
Home	Order and stock KPIs, open orders, pending claims, reorder suggestions with one-tap "Order now", recent orders.
Orders	Catalog ordering with live price summary (subtotal, discount, GST, total), order tracking with a progress bar, cancel while Processing, invoice download.
Inventory	Closing stock, reorder suggestions, stock movements, stock in/out and damage adjustments, stock statement PDF.
Retailers	Onboard retailers with GPS and KYC uploads; search, filter by KYC status, sort; tap to call.
More	Invoices, claims (submit, track, revise and resubmit), notifications, profile, support, logout.

7.3 Device behaviour

- Tokens are stored in the device keystore (Android Keystore / iOS Keychain) through [expo-secure-store](#).
- Background tracking shows an Android foreground notification ("AHCL 360 is tracking your route") so the user always knows when GPS is active.
- GPS points are buffered on the phone and uploaded in batches to save battery and data.
- Bills and KYC documents can be taken with the camera, picked from the gallery or picked as PDF files.
- The app follows the phone's light or dark theme.

8. Testing and Quality Checks

8.1 Automated API tests (17)

Area	What is tested
Authentication	OTP login, wrong OTP rejected, refresh-token rotation and reuse rejection; partner blocked from the employee panel; PAN/Aadhaar masked in responses and encrypted in the database; blocked user loses access immediately.
Employee panel	Check-in/out and monthly sheet; GPS pings, route distance, visit check-in, “one open visit” rule; leave apply and manager approval (self-approval blocked); GPS-based expense calculation, approval and reimbursement; DSO, targets, work report review, performance matrix; payslip PDF and four report types; offline sync applied exactly once.
Partner panel	Server-side pricing (self-discount ignored, GST total correct); order lifecycle → invoice (CGST = SGST) → automatic stock-in; adjustment, negative-stock rejection, reorder suggestions; retailer onboarding linked to distributor code; claim approval; access to another distributor’s stock denied; dashboards.

8.2 Build, bundle and lint checks

- Admin panel production build: passes.
- Mobile bundles for Android (3.6 MB) and iOS (3.3 MB): compile.
- Expo doctor: 21/21. Expo lint: clean.
- Smoke test of 27 API calls through the admin panel’s proxy: all HTTP 200.

Not yet verified: the screens have not been checked visually on a phone or in a browser, and background GPS and push notifications need a development build (see Section 10). These are the first recommended next steps.

9. How to Run the Project (Development)

9.1 Prerequisites

Software	Required?	Notes
Node.js 20 or newer (24 used)	Yes	https://nodejs.org . npm is included.
A code editor (VS Code)	Recommended	
Expo Go app on an Android or iOS phone	For the mobile app	From the Play Store / App Store. The phone and PC must be on the same Wi-Fi.
Docker Desktop	Optional	Only for a persistent local MongoDB. Without it the API uses a temporary in-memory database.
Android Studio emulator	Optional	Alternative to a physical phone.

9.2 Start the backend API (Terminal 1)

```
cd C:\Users\ASUS\Desktop\Project\backend
npm install
npm run dev
```

Expected output ends with: `[api] AHCL 360 API listening on http://localhost:4000/api`. The first start downloads a MongoDB binary (about 70 MB) for the in-memory database and loads the demo data. Data resets every time the API restarts.

Check it works by opening `http://localhost:4000/api/health` in a browser; it should show `{"ok":true,...}`.

9.3 Start the admin panel (Terminal 2)

```
cd C:\Users\ASUS\Desktop\Project\admin
npm install
npm run dev
```

Open `http://localhost:5173` and sign in with **9000000001 / Password@123**. The panel forwards `/api` calls to the backend on port 4000, so the API must be running.

9.4 Start the mobile app (Terminal 3)

```
cd C:\Users\ASUS\Desktop\Project\mobile
npm install
npx expo start
```

Use **npx expo start** (or **npm start**), not **npm expo start**. npm has no “expo” command.

1. A QR code appears in the terminal.
2. Android: open Expo Go and scan the QR code. iPhone: scan it with the Camera app.
3. The app finds the API automatically at your PC's Wi-Fi address on port 4000.
4. On the login screen choose **Employee** or **Channel Partner**, enter a demo mobile number and use OTP (the code is shown on screen in development) or the password.
5. When Windows Firewall asks, allow Node.js on **private networks**, otherwise the phone cannot reach the API.

If the phone still cannot connect, find the PC's IP with `ipconfig` (IPv4 Address), create `mobile/.env` with the line below and restart Expo:

```
EXPO_PUBLIC_API_URL=http://192.168.1.10:4000/api
```

9.5 Demo accounts

Password for every account: **Password@123** (or use OTP).

Mobile number	Role	Use in
9000000001	Super admin	Admin panel
9000000005	Admin (HR)	Admin panel
9000000002	Manager (2 juniors)	Employee app and admin panel
9000000003	Field executive (Sneha Das)	Employee app
9000000004	Field executive (Amit Sahoo)	Employee app
9100000001	Super distributor	Partner app
9100000002	Distributor (Mahanadi Traders)	Partner app
9100000003	Stockist	Partner app

9.6 Optional: persistent database with Docker

```
cd C:\Users\ASUS\Desktop\Project
docker compose up -d
copy backend\.env.example backend\.env
REM edit backend\.env and set: MONGO_URI=mongodb://localhost:27017/ahcl360
cd backend
npm run seed
npm run dev
```

`npm run seed -- --reset` wipes and reloads the demo data.

9.7 Run the automated tests

```
cd C:\Users\ASUS\Desktop\Project\backend
npm test
```

Expected result: `pass 17, fail 0`.

9.8 Troubleshooting

Problem	Solution
Unknown command: "expo"	Use <code>npx expo start</code> , not <code>npm expo start</code> .
Phone shows "Network unavailable" at login	Make sure the API is running, the phone is on the same Wi-Fi, the firewall allows Node.js, or set <code>EXPO_PUBLIC_API_URL</code> .
Admin panel login fails / HTTP 502	The backend is not running on port 4000. Start it first.
Port 4000 already in use	Close the other program, or start the API with a different <code>PORT</code> in <code>backend/.env</code> and update the admin proxy (<code>VITE_API_PROXY</code>) and mobile URL.
Data disappeared after restarting the API	Expected with the in-memory database. Use Docker or MongoDB Atlas (Section 9.6).

Problem	Solution
GPS tracking says “Outside office hours” or “Not checked in”	Tracking runs only after check-in and within office hours (default 09:30–18:30, Mon–Sat, IST). Office hours can be changed per user in the admin panel.
No push notifications on Android in Expo Go	Expected: remote push needs a development build (Section 10). Notifications still appear in the in-app list.
Maps blank on an Android build	Add a Google Maps API key in mobile/app.json (Section 10).

10. Steps Required for Production

10.1 Backend configuration

Copy `backend/.env.example` to `backend/.env` on the server and set at least the following:

Variable	Value
NODE_ENV	production
MONGO_URI	MongoDB Atlas or a managed MongoDB connection string
JWT_ACCESS_SECRET, JWT_REFRESH_SECRET	Two different long random strings: <code>node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"</code>
ENCRYPTION_KEY	64 hex characters: <code>node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"</code> . Back this key up: encrypted data and files cannot be read without it.
CORS_ORIGINS	The admin panel URL, e.g. <code>https://admin.ahcl.example</code>
OTP_EXPOSE / SMS_WEBHOOK_URL	<code>false</code> / your SMS provider's HTTP endpoint (receives <code>{to, message}</code>)
COMPANY_* settings	Legal name, GSTIN, address, state code, support phone and email (printed on invoices)
RATE_PER_KM_*	Company travel reimbursement rates

10.2 Hosting

1. Host the API on AWS (EC2, Elastic Beanstalk or ECS) or Google Cloud (Cloud Run / Compute Engine) behind HTTPS. Run `npm ci --omit=dev` and `npm start` under a process manager such as PM2.
2. Keep `backend/uploads` on persistent storage and back it up (or move file storage to S3/Cloud Storage).
3. Build the admin panel with `npm run build` (set `VITE_API_URL` to the API URL) and host the `admin/dist` folder on any static host (S3 + CloudFront, Firebase Hosting, Netlify, Nginx).
4. The built-in reminder scheduler runs inside the API process; with more than one API instance, run it on one instance only (set `DISABLE_JOBS=true` on the others).

10.3 Push notifications (Firebase Cloud Messaging)

1. Create a Firebase project and add Android and iOS apps with the IDs `com.ahcl.app360`.
2. Download `google-services.json` (Android) / `GoogleService-Info.plist` (iOS) into the mobile project and reference them in `app.json`.
3. Create a service-account key, place the JSON on the server and set `FIREBASE_SERVICE_ACCOUNT` to its path.
4. Run `npm install firebase-admin` in `backend`.

10.4 Mobile builds and store release

1. Replace `REPLACE_WITH_GOOGLE_MAPS_ANDROID_KEY` in `mobile/app.json` with a Google Maps Android API key.
2. Set `EXPO_PUBLIC_API_URL` to the production API URL.
3. Create an Expo account and run `npx eas-cli@latest build --profile development` to test background GPS and push on real devices.
4. Build release versions with `npx eas-cli@latest build --platform all` and submit with `npx eas-cli@latest submit`.

5. For Google Play, complete the background-location declaration (explain that routes are tracked during office hours for distance and expense calculation) and the data-safety form.

11. Known Limitations and Recommended Next Steps

Limitations

- Screens have not yet been reviewed visually on devices or in a browser.
- Background GPS and remote push notifications only work in a development or store build, not in Expo Go.
- OTP is shown on screen in development; a real SMS provider is needed for production.
- Leave quotas, travel rates, the 300 m visit radius and the performance-score weights are fixed defaults and should be confirmed with AHCL.
- Employee and partner role names, invoice numbering format and GST details are based on common Indian practice and should be checked by AHCL's accounts team.
- The project is not yet under version control (Git).

Recommended next steps

1. Initialise a Git repository and make the first commit.
2. Walk through every screen on an Android phone with the demo accounts and collect UI feedback.
3. Confirm the business rules listed above with AHCL and adjust the defaults.
4. Set up MongoDB Atlas, Firebase and an SMS provider; build a development build with EAS.
5. Pilot with a small group of field staff and one distributor before full rollout.
6. Add an analytics premium tier, more report formats and data import (Excel upload of products, distributors and employees) as needed.